CS 4122: Reinforcement Learning
Programming Assignment 3 Report
Optimizing Air Quality Monitoring using TD-Learning

Course: Reinforcement Learning
Professor: Gourav Saha
Team Members: Abhishek Gupta (se23ucse011), Samala Hanish Reddy (se23uari111), Dalip Singh Patiala (se23uari030)

Individual Technical Contributions

The assignment instruction requires the first page of `report.pdf` to contain a detailed description of the contribution made by individual team members. Documenting the work alone is not treated as a valid contribution; therefore, the entries below mention technical implementation and analysis work.

Team Member	Roll Number	Technical Contribution
Dalip Singh Patiala	se23uari030	Implemented the custom Gymnasium environment and tested state transitions for battery update, air-pollution transition, estimate update, and episode truncation.
Samala Hanish Reddy	se23uari111	Implemented Q-learning, including the state-dependent action constraint where transmission actions are allowed only when $b \geq \eta$.
Abhishek Gupta	se23ucse011	Implemented the Structural Knowledge TD / Expected-SARSA training pipeline, policy testing, policy analysis, and generation of plots for report analysis.

Submitted Files

File	Purpose
`report.pdf`	MDP formulation, algorithm explanation, testing results, plots, and policy analysis.
`GymAirQuality.py`	Custom OpenAI Gymnasium environment for the air-quality sensor transmission problem.
`train.py`	Implementation of `QLearning()` and `QLearning_StructuralKnowledge()`.
`policy1.npy`, `policy2.npy`	Final trained policies returned by QLearning and Structural Knowledge TD / Expected-SARSA respectively.

1. Problem Overview

The problem models a solar-powered air-quality sensor that observes the true normalized air pollution at its location and decides whether or not to transmit information to a central station. The central station keeps the last successfully received pollution value as its estimate. Wireless transmission is unreliable and succeeds only with probability $\lambda = 0.1$. Transmissions consume battery, while the battery is recharged by solar energy. Therefore, the sensor must plan when to save energy and when to transmit useful information.

The objective is to minimize the discounted loss caused by mismatch between the true pollution θ and the central station estimate $\hat{\theta}$. In reinforcement-learning form, this is implemented as maximizing the negative loss reward.

2. MDP Formulation

2.1 State and State Space

The state at time slot t is represented using four discrete components:

$$s_t = (\theta_t, b_t, \hat{\theta}_t, m_t)$$

Here θ_t is the true pollution index, b_t is the battery level, $\hat{\theta}_t$ is the latest estimate stored at the central station, and m_t is the maximum pollution value observed since the last successful transmission. The fourth component is required because Action 3 transmits the maximum value since the last successful transmission.

The index representation used in code is:

$$\theta_t, \hat{\theta}_t, m_t \in \{0, 1, \ldots, 50\}$$

where real pollution value = index / 50. Battery $b_t \in \{0, 1, \ldots, 10\}$. Hence the observation space is:

```
MultiDiscrete([51, 11, 51, 51])
```

2.2 Action Space

Code Action	**Assignment Action**	**Meaning**	**Feasibility**
0	Action 1	Do not transmit any value.	Always feasible.
1	Action 2	Transmit the true pollution θ_t.	Feasible only if $b_t \geq \eta$.
2	Action 3	Transmit maximum pollution m_t since the last successful transmission.	Feasible only if $b_t \geq \eta$.

Gymnasium does not directly define a state-dependent action space. Therefore, the environment declares `Discrete(3)`, while the learning code restricts the agent to valid actions using the battery condition $b_t \geq \eta$.

2.3 Reward

The assignment defines loss l_t after the possible transmission and acknowledgement in the current time slot. The reward used by the RL agent is the negative loss:

$$r_t = -l_t$$

$$l_t = |\theta_t - \hat{\theta}_t|, \quad \text{if } \theta_t \leq \hat{\theta}_t$$

$$l_t = 1.5|\theta_t - \hat{\theta}_t|, \quad \text{if } \theta_t > \hat{\theta}_t$$

The factor 1.5 penalizes under-estimation of pollution more heavily than over-estimation. Thus, the learned policy may sometimes prefer transmitting a higher maximum value rather than only the current true value.

2.4 State Transition Equations

Air pollution evolves according to the Markov matrix stored in `air.npy`. If $\theta_t = i/50$ and $\theta_{t+1} = j/50$, then the transition probability is `air[i,j]`. Solar charging is sampled from `solar.npy`, where index d corresponds to solar energy $\delta_t = d$.

If the chosen action is no transmission, the estimate remains unchanged and battery is charged by solar energy. If transmission is attempted, $\eta = 2$ battery units are consumed. If the transmission succeeds, the estimate is updated to the transmitted value; otherwise it remains unchanged.

Battery update:

$$b_{t+1} = \min(B, b_t + \delta_t), \quad \text{for Action 1}$$
$$b_{t+1} = \min(B, b_t - \eta + \delta_t), \quad \text{for Actions 2 or 3}$$

Estimate update after transmission decision:

$$\hat{\theta}_{new} = \theta_t \text{ if Action 2 succeeds; } \hat{\theta}_{new} = m_t \text{ if Action 3 succeeds; otherwise } \hat{\theta}_{new} = \hat{\theta}_t$$

Maximum pollution update:

$$m_{t+1} = \theta_{t+1}, \quad \text{if transmission succeeds}$$
$$m_{t+1} = \max(m_t, \theta_{t+1}), \quad \text{otherwise}$$

3. Custom Gymnasium Environment

The class `SensorTransmissionEnv` implements the environment in `GymAirQuality.py`. The fixed parameters used are $\lambda = 0.1$, $B = 10$, $\eta = 2$, and $\Delta = 3$. The Markovian air transition matrix is loaded from `air.npy` and the solar energy distribution is loaded from `solar.npy`.

The `reset()` function initializes θ, battery, and $\hat{\theta}$ uniformly at random over their spaces. The fourth state m is initialized as the current true pollution index θ, since at the beginning of an episode the maximum observed value since the last successful transmission can consistently start from the currently observed pollution.

The `step(action)` function performs the following sequence: validates the action against battery level, samples transmission success, updates the estimate after the transmission decision, computes reward as negative loss, samples solar energy, updates the battery, samples next pollution using `air.npy`, updates m, increments the time-slot counter, and returns the Gymnasium tuple.

The task is continuous, but for training it is truncated after 288 time slots. This represents one day because each slot is 5 minutes long, so $288 \times 5 = 1440$ minutes. The environment always returns `terminated = False`, `truncated = True` only after the 288th time slot, and `info = {}`.

4. Training Algorithms

4.1 Q-Learning

The first algorithm is standard off-policy Q-learning. The behavior policy is epsilon-greedy over valid actions, while the update target uses the greedy value of the next state. The Q-learning update used is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \beta \max_{a'} Q(s',a') - Q(s,a) \right]$$

Only valid actions are considered in both action selection and $\max_{a'}$. If $b < \eta$, the only valid action is Action 1. If $b \geq \eta$, all three actions are valid. The final output is not the Q-table; it is the greedy policy array `policy1.npy` with shape $(51, 11, 51, 51)$.

4.2 Structural Knowledge TD / Expected-SARSA Implementation

The second algorithm follows the Expected-SARSA TD idea. Instead of using only the maximum next-state Q-value, it computes the expected next-state Q-value under an epsilon-greedy policy over valid actions. This reduces the noise of relying on a single sampled next action while still using TD learning.

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \beta \sum_{a'} \pi(a'|s') Q(s',a') - Q(s,a) \right]$$

In this implementation, the structural knowledge used is the state-dependent feasible-action structure of the problem. Since transmission actions are impossible when the battery level is below η, both the TD target and the exploration policy are restricted to feasible actions only. For states with sufficient battery, the expected update is computed over all valid actions using epsilon-greedy probabilities. This makes the update consistent with the battery-constrained structure of the MDP and prevents invalid transmission choices from affecting learning.

The structural cases considered in the implementation are documented below. The code considers only these cases.

Case	**Condition**	**Actions considered / policy probabilities**
1	$b < \eta$	Only Action 1 is valid. Therefore $\pi(\text{Action } 1|s) = 1$ and transmission actions are not sampled or used in the TD target.
2	$b \geq \eta$ and a unique greedy action exists	All actions {1,2,3} are valid. The greedy action gets probability $1 - \epsilon + \epsilon/|A(s)|$, and each other valid action gets $\epsilon/|A(s)|$.
3	$b \geq \eta$ and the best action is selected after comparing valid Q-values	The Expected-SARSA target is computed over only the valid next actions. This prevents invalid transmission choices when the next battery level is below η.

4.3 Training and Testing Setup

Both algorithms were trained for 10000 episodes using the original training run. The discount factor was $\beta = 0.98$ and the learning rate was $\alpha = 0.1$. The exploration probability was decayed over training to move from exploration toward exploitation.

Testing was performed during training. Every 100 training episodes, the current greedy policy was tested and the average sum of rewards was saved. This produced 100 testing points for each algorithm. The saved arrays used in this report are `qlearning_scores.npy` and `structural_scores.npy`. Since reward is negative loss, a larger value, meaning a value closer to zero, is better.

5. Testing Results and Learning Curve

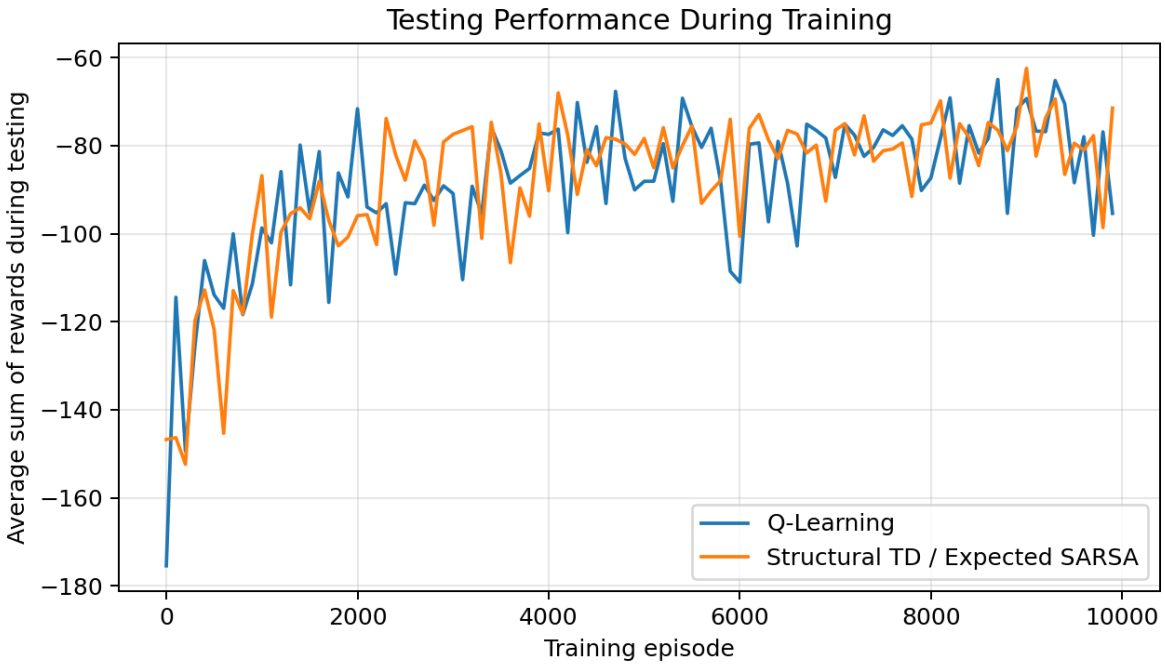

Figure 1: Average sum of testing rewards versus training episode for both TD-based learning algorithms.

Metric	**Q-Learning**	**Structural TD / Expected-SARSA**
Best testing score	-64.956	-62.425
Episode of best score	8700	9000
Final saved testing score	-95.374	-71.459
Mean of last 10 testing scores	-79.747	-78.266

From the plotted curve, Q-learning reaches the higher reward region slightly earlier in the training process, so it can be interpreted as learning faster in this run. However, the Structural TD / Expected-SARSA run gives a slightly better best testing score and a better final saved testing score. Thus, Q-learning is faster in early improvement, while the structural/expected update is competitive and slightly better on the final recorded test values.

6. Policy Action Distribution

Policy	**Action 1: no transmit**	**Action 2: transmit theta**	**Action 3: transmit max**
`policy1.npy`	1401746 (96.07%)	35765 (2.45%)	21650 (1.48%)
`policy2.npy`	1396925 (95.73%)	37713 (2.58%)	24523 (1.68%)

The learned policies choose Action 1 in most states. This is expected because many states either do not have enough battery to transmit, or the expected benefit of transmission is small compared to saving energy for future slots. The battery-dependent action constraint was respected: for battery levels $b = 0$ and $b = 1$, the policies choose Action 1 only.

7. Pattern for Action 1: Do Not Transmit

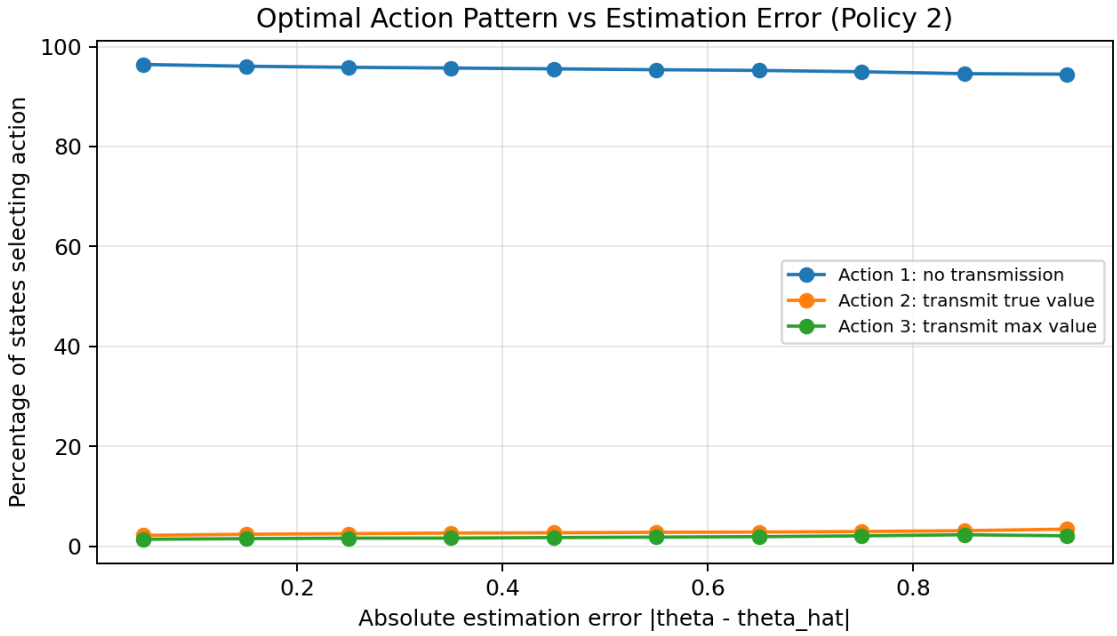

Figure 2: Distribution of optimal actions in `policy2.npy` as the absolute estimation error changes.

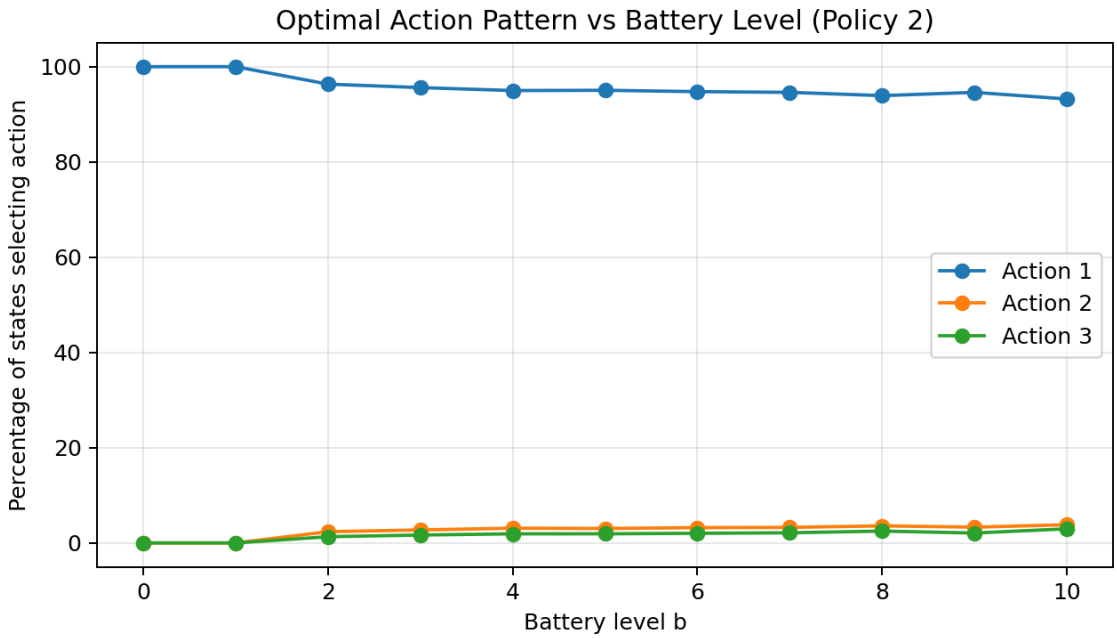

Figure 3: Distribution of optimal actions in `policy2.npy` as battery level changes.

Action 1 is optimal most often when the battery is low or when the estimation error is small. When $b < \eta$, transmission is infeasible, so Action 1 must be selected. Even when $b \geq \eta$, Action 1 can still be optimal if $|\theta - \hat{\theta}|$ is small because transmitting would consume battery for only a marginal reduction in estimation error. The policy therefore saves energy for future time slots where the mismatch may become larger.

8. Pattern for Action 3: Transmit Maximum Pollution Since Last Success

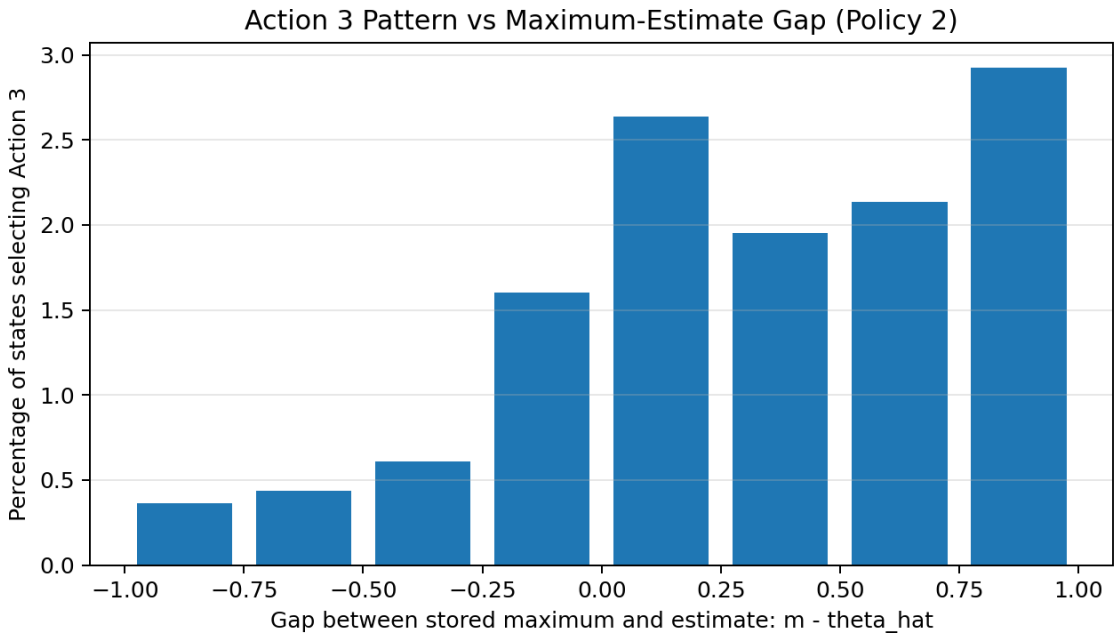

Figure 4: Percentage of states choosing Action 3 as the gap $m - \hat{\theta}$ changes.

Action 3 becomes useful when the maximum pollution observed since the last successful transmission is substantially higher than the estimate stored at the central station. This pattern is consistent with the asymmetric loss function. Under-estimation is penalized by 1.5 times the absolute error, so when the stored estimate is low and the observed maximum is high, transmitting the maximum value can reduce the risk of future under-estimation.

Action 3 is still chosen less frequently than Action 1 because it requires enough battery and also faces unreliable wireless success probability $\lambda = 0.1$. Therefore, the policy uses it selectively, mainly when the potential penalty from under-estimation is large enough to justify spending battery.

9. Conclusion

The assignment problem was formulated as a finite MDP with four state components: true pollution, battery level, central-station estimate, and maximum pollution since the last successful transmission. A custom OpenAI Gymnasium environment was implemented with the specified parameters and data files. Two TD-based learning algorithms were implemented from scratch: Q-learning and Structural Knowledge TD / Expected-SARSA. Both algorithms account for the state-dependent action space and return the final greedy policy as a NumPy array.

Testing was performed during training and the saved testing rewards were used for comparison. In this run, Q-learning showed faster early improvement, while Structural TD / Expected-SARSA achieved a slightly better best and final testing score. Policy analysis showed that Action 1 is mainly optimal when battery is low or estimation error is small, while Action 3 is mainly useful when the previously observed maximum pollution is much larger than the central-station estimate.

References

1. CS 4122 Reinforcement Learning, Programming Assignment 3 PDF.

2. Lecture 25-26 slides on TD control, SARSA variants, Expected-SARSA, and Q-learning.